

Brain-age training: scenario D (default)

Technical description of the default EEG brain-age fine-tune, `labram/configs/defaults/finetune_tuab_age.json`. It is scenario **D** of the October 2026 ablation: LaBraM-base fine-tuned on TUAB with a common average reference on the input. Background on the task and data: `age_regression.md`; next steps: `brain_age_improvement_plan.md`.

1. Result

Best epoch (selected on validation case MAE), from ClearML task `age_D_eval5min_car` (SageMaker, `ml.g5.2xlarge`):

Split	Case MAE (years)	Window MAE (years)	R^2 (case)	Pearson r (case)	Age-bias slope	Bias-corrected MAE
Validation	7.77	8.61	0.61	0.78	−0.39	6.22
Test	8.30	9.09	0.65	0.81	−0.40	6.11

"Case" means one prediction per recording (window predictions averaged); "window" scores each 10 s window on its own. For reference, predicting the training mean scores about 12.8 (validation) and 14.9 (test), and the previous best fine-tune scored 8.31 / 9.04.

2. Data

Corpus. TUH Abnormal EEG Corpus (TUAB) v3.0.0, windows preprocessed by `dataset_maker/make_TUAB.py`: 0.1–75 Hz band-pass, 50 Hz notch, resampled to 200 Hz, in μV , cut into consecutive non-overlapping 10 s windows of 23×2000 samples. Window k of a recording covers $[10k, 10k + 10)$ s.

Label. The patient's age in years, parsed from the EDF header (Age: field) and joined onto the windows by recording; ages outside 1–89 (TUH's Age:999 redaction) are excluded.

Split. Subject-disjoint and seeded (`processed/age_split.json`, seed 12345, 20 % of training subjects held out for validation); the test set is TUAB's official evaluation set. Normal and abnormal recordings are both included.

Window selection (`labram/data/window_selection.py`). The first and last 60 s of every recording are dropped (`trim_start_sec = trim_end_sec = 60`). Training uses every remaining window; validation and test use only the first 5 minutes after the trim (`eval_minutes = 5`), i.e. 30 windows per recording.

Split	Subjects	Recordings	Windows used
Train	1,650	2,145	268,513
Validation	412	556	16,680
Test	251	274	8,220

Target normalization. Each window's age y_i is z-scored with the mean and standard deviation of the selected training windows:

$$z_i = \frac{y_i - \mu}{\sigma}, \quad \mu = 48.88 \text{ years}, \quad \sigma = 17.82 \text{ years}$$

3. Input and feature building

3.1 Input tensor

A window is scaled to units of 100 μV and split into 1 s patches of 200 samples per channel:

$$x \in \mathbb{R}^{C \times A \times T}, \quad C=23 \text{ channels}, \quad A=10 \text{ patches}, \quad T=200 \text{ samples}$$

Channels (TUH referential 10-20 montage, including the ear electrodes A1/A2 and the temporal T1/T2): FP1, FP2, F3, F4, C3, C4, P3, P4, O1, O2, F7, F8, T3, T4, T5, T6, A1, A2, FZ, CZ, PZ, T1, T2.

3.2 Common average reference (the change that defines scenario D)

At every sample, the mean over all channels is subtracted (`labram_plus.enabled = true`, `common_average_reference = true`, `z_score_patches = false`):

$$\tilde{x}_{c,a,t} = x_{c,a,t} - \frac{1}{C} \sum_{c'=1}^C x_{c',a,t}$$

This removes activity shared by every electrode (the recording reference and common-mode noise) while keeping absolute amplitude, which carries age information; per-patch z-scoring is therefore off. The step has no parameters and runs inside the model (`NeuralTransformer.maybe_preprocess_input`), so training and evaluation always see identically referenced input.

3.3 Patch embedding

Each patch $\tilde{x}_{c,a} \in \mathbb{R}^{200}$ passes through `TemporalConv`, three 1-D convolutions shared by all channels and patches:

Layer	Filters	Kernel / stride / padding	Then	Output
conv1	8	15 / 8 / 7	GroupNorm(4) + GELU	8×25
conv2	8	3 / 1 / 1	GroupNorm(4) + GELU	8×25
conv3	8	3 / 1 / 1	GroupNorm(4) + GELU	8×25

The 8×25 output is flattened into a token $u_{c,a} \in \mathbb{R}^d$ with $d=200$.

3.4 Token sequence

A learned class token is prepended, and every token receives a learned spatial embedding indexed by its electrode's position $\pi(c)$ in the standard 10-20 table, plus a learned temporal embedding for its patch index a :

$$h_{c,a}^{(0)} = u_{c,a} + P_{\pi(c)} + E_a, \quad h_{\text{cls}}^{(0)} = t_{\text{cls}} + P_0$$

The sequence holds $1 + C \cdot A = 231$ tokens. The spatial table P has 129 rows and the temporal table E 16 rows (rows 0-9 are used for a 10 s window).

3.5 Transformer encoder

Twelve pre-norm blocks of width $d=200$ with 10 heads ($d_h=20$), an MLP of width 800, layer scale γ (initialized to 0.1) and stochastic depth increasing linearly from 0 to 0.1 across blocks:

$$h' = h + \text{DropPath}(\gamma_1 \odot \text{MHSA}(\text{LN}(h))), \quad h'' = h' + \text{DropPath}(\gamma_2 \odot \text{MLP}(\text{LN}(h')))$$

Attention uses LayerNorm on queries and keys (QK-norm), no QKV bias and no relative position bias; every token attends to every other token across all channels and time:

$$\text{Attn}(Q, K, V) = \text{softmax}\left(\frac{\text{LN}(Q)\text{LN}(K)^\top}{\sqrt{d_h}}\right)V$$

3.6 Pooling and regression head

The class token is discarded; the 230 patch tokens of the last block are averaged and normalized, giving the window's **feature vector** $f \in \mathbb{R}^{200}$, which a linear head maps to the normalized age:

$$f = \text{LN}_{\text{fc}}\left(\frac{1}{CA} \sum_{c=1}^C \sum_{a=1}^A h_{c,a}^{(12)}\right), \quad \hat{z} = w^\top f + b$$

Parameter count: 5,820,137, all trainable (patch embedding and embeddings 26,576; each block 482,480; fc_norm, head and temporal embedding 3,801). fc_norm and the head are newly initialized; everything else loads from checkpoints/labram-base.pth.

4. Training loss

Scenario D minimizes a single term. With the residual in units of σ ,

$$r_i = \hat{z}_i - z_i = \frac{\hat{y}_i - y_i}{\sigma}$$

each window contributes a Huber loss with $\delta=1$ (`loss.huber_delta`):

$$\ell_\delta(r) = \begin{cases} \frac{1}{2}r^2, & |r| \leq \delta \\ \delta\left(|r| - \frac{1}{2}\delta\right), & |r| > \delta \end{cases}$$

The total loss is the mean over the $N=64$ windows of a batch:

$$\mathcal{L}_{\text{total}}(\theta) = \frac{1}{N} \sum_{i=1}^N \ell_\delta\left(f_\theta(x_i) - \frac{y_i - \mu}{\sigma}\right) = \mathcal{L}_{\text{Huber}}$$

The code's composite criterion would add amplitude, phase and quantization terms ($w_{\text{amp}}=1$, $w_{\text{phase}}=0.1$, $w_{\text{emb}}=1$) only when `model.codebook_reg.enabled` is true; in scenario D they are absent, so $\mathcal{L}_{\text{total}} = \mathcal{L}_{\text{Huber}}$ exactly.

Because $\delta\sigma=17.82$ years, errors below about 18 years are penalized quadratically and larger ones linearly. The gradient with respect to the prediction is capped, so a mislabeled or artifact-heavy window cannot dominate a batch:

$$\frac{\partial \ell_\delta}{\partial \hat{z}} = \begin{cases} r, & |r| \leq 1 \\ \text{sign}(r), & |r| > 1 \end{cases}$$

Error (years)	r	Huber loss	Gradient	Half-MSE loss	MSE gradient
5	0.281	0.039	0.281	0.039	0.281
15	0.842	0.354	0.842	0.354	0.842
30	1.683	1.183	1	1.417	1.683
60	3.366	2.866	1	5.667	3.366

5. Optimization

Optimizer. AdamW, $\beta = (0.9, 0.999)$, $\varepsilon = 10^{-8}$, batch 64, 15 epochs of 4,195 steps (62,925 steps), mixed precision (fp16 autocast with dynamic loss scaling), no gradient clipping. Weight decay $\lambda = 0.05$ is applied to weight matrices only (biases, norms, layer-scale vectors and the class, spatial and temporal embeddings get none) and is not part of the loss:

$$\theta_{s+1} = \theta_s - \eta_{\ell}(s) \frac{\hat{m}_s}{\sqrt{\hat{v}_s + \varepsilon}} - \eta_{\ell}(s) \lambda \theta_s$$

Learning-rate schedule. Linear warmup over $S_w = 8,390$ steps (2 epochs) from 10^{-6} to $\eta_{\max} = 10^{-4}$, then cosine decay to $\eta_{\min} = 10^{-6}$ over the remaining $S_c = 54,535$ steps:

$$\eta(s) = \eta_{\min} + \frac{1}{2}(\eta_{\max} - \eta_{\min}) \left(1 + \cos \frac{\pi(s - S_w)}{S_c} \right), \quad s \geq S_w$$

Layer-wise decay. Layer group ℓ (0 = patch embedding, 1-12 = blocks, 13 = head) steps with a scaled rate, $\eta_{\ell}(s) = 0.65^{13-\ell} \eta(s)$:

Group	Parameters	Scale	Peak learning rate
13	head, fc_norm, temporal embedding	1.000	1.0×10^{-4}
12	block 11	0.650	6.5×10^{-5}
10	block 9	0.275	2.7×10^{-5}
7	block 6	0.075	7.5×10^{-6}
4	block 3	0.021	2.1×10^{-6}
1	block 0	0.0057	5.7×10^{-7}
0	patch embedding, spatial embedding, class token	0.0037	3.7×10^{-7}

The lower half of the network therefore moves 20–270 times slower than the head, which keeps the pretrained low-level features largely intact.

6. Evaluation and model selection

After every epoch the model scores validation and test without dropout. Predictions are de-normalized, averaged over the $K=30$ windows of each recording, and compared with the recording's age over R recordings:

$$\hat{y}_i = \mu + \sigma \hat{z}_i, \quad \hat{y}_j^{\text{rec}} = \frac{1}{K} \sum_{k=1}^K \hat{y}_{j,k}, \quad e_j = \hat{y}_j^{\text{rec}} - y_j$$

$$\text{MAE} = \frac{1}{R} \sum_{j=1}^R |e_j|, \quad \text{RMSE} = \sqrt{\frac{1}{R} \sum_{j=1}^R e_j^2}, \quad R^2 = 1 - \frac{\sum_j e_j^2}{\sum_j (y_j - \bar{y})^2}$$

Brain-age models shrink toward the cohort mean, so two diagnostics fit the residual linearly against the true age by least squares, $e_j \approx a y_j + b$: the **age-bias slope** is a (0 is ideal), and the **bias-corrected MAE** is

$$\text{MAE}_{\text{corr}} = \frac{1}{R} \sum_{j=1}^R |e_j - (a y_j + b)|$$

The checkpoint with the lowest validation case MAE is kept (checkpoint-best.pth), and test metrics are reported at that epoch. In scenario D the best epoch is the second (index 1).

7. How scenario D was chosen

All runs share the split, the trims, 15 epochs at peak learning rate 10^{-4} with 2 warmup epochs, and the starting checkpoint; MAE is case-level at each run's best validation epoch.

Run	Change vs. B	Validation MAE	Test MAE
A	whole-recording evaluation instead of 5 minutes	8.29	9.03
B	baseline: 5-minute evaluation (2 of 15 epochs at time of writing)	8.49	8.91
C	30 s windows	8.24	9.35
C2	layer decay 0.5, drop path 0.2, dropout 0.1, weight decay 0.1, clip 1.0	12.47	13.52
D	common average reference	7.77	8.30
D2	L1 loss instead of Huber	8.36	9.32
C2 + head only	only the regression head trainable	12.77	14.78
C2 + last block	last block and head trainable	12.50	13.64

D is the only change with a clear gain. Restricting adaptation to the top of the network (C2 and the two partial fine-tunes) underfits at this learning rate.

8. Reproducing

Local, single GPU (each run writes to a fresh timestamped output directory):

```
python -m labram.runs.run_finetune \
  --config labram/configs/defaults/finetune_tuab_age.json \
  --set data.data_path=/data/datasets/EEG-public/TAUB/TUH_Abnormal/v3.0.0/edf \
  clearml.enabled=true
```

SageMaker (spot, data streamed from S3):

```
python -m labram.runs.submit_sagemaker \
  --config labram/configs/defaults/finetune_tuab_age.json --detach \
  --set sagemaker.enabled=true sagemaker.use_spot=true sagemaker.max_wait_min=2880 \
  sagemaker.input_mode=FastFile \
  sagemaker.role=arn:aws:iam::574441342949:role/SageMakerExecutionRole \
  data.data_path=s3://eeg-data-public/TUH_Abnormal/v3.0.0/edf/processed/ \
  output.output_dir= output.log_dir= clearml.enabled=true
```

9. Known limitations

- **Early overfitting.** Validation MAE is best at the second epoch; by the last epoch training MAE falls to about 1.2 years while validation drifts to 7.9. Best-checkpoint selection protects the reported result; random crops and augmentation are the planned remedy.
- **Regression to the mean.** The age-bias slope is about -0.4 : older patients are predicted too young and younger ones too old.
- **Mixed cohort.** Training and evaluation include abnormal recordings, so results are not directly comparable with normal-only benchmarks (6.60 years, Gemein et al. 2024).

- **Spot interruptions.** Checkpoints are not synced to S3, so an interrupted SageMaker spot job restarts from the pretrained weights.

10. Implementation map

Concern	File
Configuration	labram/configs/defaults/finetune_tuab_age.json
Age labels and split	labram/data/tuh_metadata.py, labram/data/age_splits.py
Window selection	labram/data/window_selection.py
Loader and target normalization	labram/data/tuh_datasets.py
Common average reference	labram/data/preprocess.py
Model	labram/models/neural_transformer.py, labram/layers/
Loss	labram/losses/regression.py
Layer-wise learning rates	labram/optim_factory.py
Training and evaluation loop	labram/train/train_finetune.py
Metrics	labram/utils/regression_metrics.py